

similar, but have differences in escapes, meta-characters, or special operators.

GNU Grep has two regular expression feature sets: *Basic* and *Extended*. In basic regular expressions, the meta-characters '?', '+', '{', '|', '(', and ')' lose their special meaning (whose uses are described later in this article). As mentioned below, to switch to using extended regular expressions, you need to add the option *-E* to the Grep command.

It is customary to enclose the regular expression in single quotation marks, to prevent the shell (Bash or others) from trying to interpret and expand the expression before launching the Grep process. For example, if a pair of back-ticks in the regexp is not quoted, it would result in the text between the back-ticks being executed as a Bash sub-process—and if this happens to be a valid command, the text returned by it takes the regular expression's place in the command-line parameters given to Grep! Not at all what we want.

Again, due to shell behaviour, you can also enclose the regex in double quotes—in this case, you can use environment variables in the regex, and the shell will substitute them before calling Grep. This can be very useful, depending on what you're trying to do—or it could turn out to be a nuisance. Remember the difference in behaviour.

Basic usage

Now let's go on to some practical examples of using Grep. To better understand the results, I've created a simple text file on which we will run our Grep searches; the file contains the following lines:

```
Hi
this
is test file
to carry out few regular expressions
practical with grep
123 456
Abcd
ABCD
```

Case-insensitive search (*grep -i*):

```
[manish@clone ~]$ grep -i 'abcd' testfile
Abcd
ABCD
```

As you can see, the *-i* flag causes a search for 'abcd' to return matches that have different cases for the characters from what the search string does.

Whole-word search (*grep -w*):

```
[manish@clone ~]$ grep -w 'test' testfile
is test file
```

This type of search only returns lines where the sought-for string is a whole word and not part of a larger word.

Recursively search through sub-folders (*grep -r <pattern>*):

<path>):

```
[manish@clone ~]$ grep -r '456' /root/
/root/testfile:Year is 2010
```

Inverted search (*grep -v*):

```
[manish@clone ~]$ grep -v 'practical' testfile
Hi
this
is test file
to carry out few regular expressions
123 456
Abcd
ABCD
```

This prints all the lines in the file, *except* the line that contains the word 'practical'.

An interesting relative is the *-L* flag (you can also use *--files-without-match*), which outputs the names of files that do NOT contain matches for your search pattern. The matches for your search pattern are not themselves printed, only the names are.

```
[manish@clone ~]$ grep -r -L "Network" /var/log/*
/var/log/anaconda.log
/var/log/anaconda.syslog
/var/log/audit/audit.log
/var/log/boot.log
/var/log/boot.log.1
...
```

The 'opposite' flag to *-L* is *-l* or *--files-with-matches*, which prints out (only) the names of files that do contain matches for your search pattern.

Print additional (trailing) context lines after match (*grep -A <NUM>*):

```
[manish@clone ~]$ grep -A1 '123' testfile
123 456
```

```
Abcd
```

For each line that matches the search, Grep prints the matching line, as well as the next one line after the match. Varying the number provided to *-A* changes the number of additional lines that are in the output.

Print additional (leading) context lines before match (*grep -B <NUM>*):

```
[manish@clone ~]$ grep -B2 'Abcd' testfile
practical with grep
```

```
123 456
```

```
Abcd
```